HOLOMECHANIX

Interactive Spatial 3D Dismantling & Engineering Learning Platform

MASTER DEVELOPMENT PROMPT

A 63-section, phase-by-phase specification for building a webcam-driven, hand-tracked 3D engineering laboratory using Python, OpenCV, MediaPipe and a 3D engine.

MASTER DEVELOPMENT PROMPT

Project Name

HoloMechanix — Interactive Spatial 3D Dismantling & Engineering Learning Platform

Build a complete Python-based interactive 3D visualization and dismantling platform inspired by futuristic spatial interfaces such as Tony Stark's workshop interface.

The application must allow a user to stand in front of a webcam and interact with detailed 3D models of objects such as:

- Cars
- Motorcycles
- Smartphones
- Engines
- Mechanical systems
- Electronics
- Machines
- Other educational models

The user should be able to use both hands and all 10 fingers to manipulate the virtual object in real time.

The system should support:

- Object rotation
- Zoom
- Translation
- Component selection
- Component highlighting
- Component movement
- Component removal
- Component replacement
- Exploded-view visualization
- Step-by-step dismantling
- Reassembly
- Component inspection
- Component information
- Interactive animations
- Mechanical movement
- Educational explanations
- Multiple 3D models
- Model switching
- Gesture-controlled UI
- Voice commands as an optional feature
- Camera-based spatial interaction

The final result should feel like a futuristic engineering laboratory rather than a normal website or basic OpenCV demo.

IMPORTANT DEVELOPMENT RULE

Do NOT attempt to generate the entire application in one step.

Build the project incrementally.

Every stage must:

1. Have a clear objective.
2. Create only the required files.
3. Implement the feature.
4. Run the application.
5. Test the feature.
6. Fix errors.
7. Explain what was implemented.
8. Wait for the developer/user to confirm completion before moving to the next major stage.

Never skip foundational stages.

Never create fake functionality just to make the application appear complete.

If a feature cannot realistically be implemented with the selected technology, clearly explain the limitation and implement the closest technically valid alternative.

1. CORE VISION

The application should behave like an interactive virtual engineering laboratory.

Example:

The user loads:

2025 Sports Motorcycle

The motorcycle appears as a 3D model.

The user raises both hands.

The system detects all visible fingers.

The user performs a gesture.

The motorcycle rotates.

The user points at the engine.

The engine becomes highlighted.

The user performs a pull gesture.

The engine moves outward from the motorcycle.

The surrounding components remain in place.

The user can inspect the engine.

The user can rotate the engine.

The user can zoom into the engine.

The user can select individual components.

Example:

Engine → Cylinder Head → Spark Plug → Piston → Connecting Rod → Crankshaft.

The application should display information about each selected component.

The user should also be able to reverse the operation and reassemble the object.

2. TECHNOLOGY STACK

Use Python as the primary programming language.

Recommended stack:

Computer Vision

- Python
- OpenCV
- MediaPipe
- NumPy

Use MediaPipe Hands or the latest stable hand-landmark solution available in the environment.

The system should detect:

- Left hand
- Right hand
- Up to 10 fingers
- 21 landmarks per hand
- Finger states
- Palm orientation
- Hand position
- Finger positions

3. 3D ENGINE

Use a proper 3D rendering engine.

Preferred architecture:

Option A

Python + Open3D

Use Open3D for:

- 3D model rendering
- Camera
- Mesh loading
- Point clouds
- Object manipulation
- Visualization

Option B

Python + Panda3D

Use Panda3D if Open3D becomes insufficient for:

- advanced interaction
- animations
- scene hierarchy
- lighting
- camera control

Option C

Python backend + Three.js frontend

If the Python-only architecture becomes restrictive, use:

Python:

- OpenCV
- MediaPipe
- FastAPI

Frontend:

- Three.js
- React
- WebGL

Use WebSockets for real-time gesture data.

Choose the architecture based on technical feasibility rather than blindly following the stack.

Document the decision.

4. MODEL FORMAT

Support:

- GLB
- GLTF
- OBJ
- FBX where technically possible
- STL for mechanical components

Prefer GLB/GLTF for production because they preserve:

- meshes
- materials
- textures
- hierarchy
- animations

Every model should have a structured component hierarchy.

Example:

```
CAR
├── Body
├── Hood
├── Doors
├── Wheels
├── Engine
│   ├── CylinderHead
│   ├── Pistons
│   ├── Crankshaft
│   ├── SparkPlugs
│   └── Alternator
├── Transmission
├── Battery
├── Suspension
├── Brakes
└── Exhaust
```

5. PROJECT ARCHITECTURE

Create a professional modular architecture.

Recommended structure:

```
HoloMechanix/
│
├── app/
│   ├── main.py
│   │
│   ├── vision/
│   │   ├── camera.py
│   │   ├── hand_tracker.py
│   │   ├── finger_tracker.py
│   │   ├── gesture_engine.py
│   │   └── spatial_mapper.py
│   │
│   ├── rendering/
│   │   ├── scene.py
│   │   ├── camera.py
│   │   ├── lighting.py
│   │   ├── model_loader.py
│   │   └── renderer.py
│   │
│   ├── interaction/
│   │   ├── selection.py
│   │   ├── manipulation.py
│   │   ├── grabbing.py
│   │   ├── rotation.py
│   │   ├── scaling.py
│   │   └── dismantling.py
│   │
│   ├── simulation/
│   │   ├── component.py
│   │   ├── constraints.py
│   │   ├── mechanisms.py
│   │   └── animation.py
│   │
│   ├── models/
│   │   ├── registry.py
│   │   ├── metadata.py
│   │   └── hierarchy.py
│   │
│   ├── education/
│   │   ├── component_info.py
│   │   ├── explanations.py
│   │   └── learning_mode.py
│   │
│   ├── ui/
│   │   ├── hud.py
│   │   ├── menus.py
│   │   └── overlays.py
│   │
│   └── config/
│       └── settings.py
│
├── assets/
│   ├── models/
│   ├── textures/
│   ├── environments/
│   └── animations/
│
├── data/
│   ├── components/
│   ├── models/
│   └── gestures/
│
├── tests/
│
```

```
├── scripts/
│
├── requirements.txt
├── README.md
├── .env.example
└── .gitignore
```

Modify this structure if required by the selected rendering engine.

6. DEVELOPMENT PHASES

Implement the system in the following phases.

Do NOT skip phases.

PHASE 0 — ENVIRONMENT SETUP

First create the project.

Verify:

```
python --version
pip --version
```

Create a virtual environment:

```
python -m venv .venv
```

Activate it.

Windows:

```
.venv\Scripts\activate
```

Install the initial dependencies.

Create:

```
requirements.txt
```

Include only dependencies actually required.

Test imports.

Create:

```
app/main.py
```

Run:

```
python app/main.py
```

The application should start successfully.

PHASE 1 — WEBCAM SYSTEM

Implement webcam capture using OpenCV.

Requirements:

- Open webcam
- Display live video
- Detect camera resolution
- Show FPS

- Allow ESC to exit
- Handle camera failure
- Automatically release camera resources

Create:

```
vision/camera.py
```

The camera abstraction should provide:

```
start()
read()
get_fps()
stop()
```

Do not mix camera code with gesture code.

PHASE 2 — HAND TRACKING

Integrate MediaPipe.

Detect:

- Left hand
- Right hand
- 21 landmarks per hand

Display landmarks over the webcam.

Show:

```
LEFT HAND
21 LANDMARKS
RIGHT HAND
21 LANDMARKS
```

Calculate:

- Palm center
- Wrist
- Thumb tip
- Index tip
- Middle tip
- Ring tip
- Pinky tip

Create:

```
hand_tracker.py
```

PHASE 3 — 10-FINGER DETECTION

Build a robust finger-state detector.

Detect whether each finger is:

- Open
- Closed
- Partially open

Represent the state:

```
{
    "left": {
        "thumb": True,
        "index": True,
        "middle": False,
        "ring": False,
        "pinky": False
    },
    "right": {
        "thumb": True,
        "index": True,
        "middle": True,
        "ring": True,
        "pinky": True
    }
}
```

Calculate:

```
finger_count
```

from 0–10.

Do NOT use simple distance-only detection.

Account for:

- hand rotation
- perspective
- different hand sizes
- left/right hand differences

PHASE 4 — GESTURE ENGINE

Create a gesture recognition system.

Initial gestures:

Open Palm

All fingers extended.

Action:

Show interaction mode.

Fist

Most fingers closed.

Action:

Pause interaction.

Point

Index finger extended.

Action:

Select component.

Pinch

Thumb + index finger close.

Action:

Grab object.

Two-Hand Pinch

Both hands pinch.

Action:

Scale or manipulate object.

Two-Finger Rotation

Use both hands.

Action:

Rotate model.

Pull Apart

Both hands move away from each other.

Action:

Explode selected assembly.

Push Together

Both hands move toward each other.

Action:

Reassemble.

Swipe

Hand moves rapidly left/right.

Action:

Switch model or menu.

Point + Pinch

Point to component and pinch.

Action:

Grab component.

PHASE 5 — GESTURE STABILITY

Raw hand tracking will produce noisy results.

Implement:

- smoothing
- exponential moving average
- landmark filtering
- gesture hysteresis

- debounce
- confidence thresholds

Example:

A gesture must remain stable for:

```
100–250 ms
```

before being accepted.

Do not allow random camera noise to trigger dismantling.

PHASE 6 — SPATIAL COORDINATE SYSTEM

Create a mapping between:

```
2D camera coordinates
```

and:

```
3D scene coordinates
```

The system should map:

```
screen X → world X
screen Y → world Y
hand depth → approximate world Z
```

Because a normal webcam cannot directly provide perfect world-space depth, use:

- MediaPipe depth estimation
- relative hand size
- palm geometry
- temporal tracking

Document that this is an approximation.

Do not claim true spatial tracking without depth hardware.

PHASE 7 — 3D RENDERING

Create the 3D scene.

Implement:

- Perspective camera
- Ambient lighting
- Directional lighting
- Shadows where supported
- Ground plane
- Environment
- Grid
- Object origin
- Camera controls

The default scene should look like a futuristic engineering laboratory.

Use a dark background.

Add subtle:

- grid

- glowing outlines
- transparent panels
- technical HUD
- component labels

Avoid excessive visual effects that reduce usability.

PHASE 8 — FIRST 3D MODEL

Start with ONE simple model.

Do not start with a complete car.

Use something like:

```
Mechanical Gearbox
```

or:

```
Engine
```

The model should contain multiple independent components.

For example:

```
Engine
├── Block
├── CylinderHead
├── SparkPlug
├── Piston
├── ConnectingRod
├── Crankshaft
└── Flywheel
```

Load the model.

Display it correctly.

Allow:

- rotate
- zoom
- pan

PHASE 9 — COMPONENT HIERARCHY

Every model must have a semantic component hierarchy.

Create a metadata format.

Example:

```
{
    "engine": {
        "name": "Engine",
        "children": [
            "cylinder_head",
            "piston",
            "crankshaft"
        ]
    }
}
```

Each component should contain:

```
{
    "id": "spark_plug_01",
    "name": "Spark Plug",
    "category": "Ignition",
    "description": "...",
    "function": "...",
    "parent": "cylinder_head",
    "removable": true,
    "movement_axis": [0, 1, 0]
}
```

PHASE 10 — COMPONENT SELECTION

Implement pointing-based selection.

User points at a component.

The system should:

1. Determine the 3D ray from the camera through the finger position.
2. Perform ray casting.
3. Detect the intersected component.
4. Highlight it.
5. Display its name.

Example:

```
SELECTED COMPONENT
Spark Plug
──────────────
Function:
Ignites the air-fuel mixture.
System:
Ignition
Parent:
Cylinder Head
```

PHASE 11 — COMPONENT HIGHLIGHTING

Selected components should visually change.

Possible effects:

- outline
- glow
- transparency of surrounding components
- color change
- pulsing boundary

When a component is selected:

```
Selected component = highlighted
Other components = slightly dimmed
```

PHASE 12 — GRAB SYSTEM

Implement pinch-based grabbing.

Gesture:

```
Thumb + Index Finger
```

When the user pinches over a selected component:

```
component becomes attached to cursor
```

The user can move it around.

Implement:

- grab
- move
- release

Use smoothing.

PHASE 13 — 3D MODEL ROTATION

Implement two-hand rotation.

Example:

User places hands on opposite sides.

Move right hand clockwise.

The model rotates.

Support:

- X rotation
- Y rotation
- Z rotation

Use relative hand movement.

PHASE 14 — SCALE / ZOOM

Implement two-hand distance scaling.

Hands move apart:

```
zoom in
```

Hands move together:

```
zoom out
```

Alternatively use:

```
pinch + hand distance
```

Avoid sudden jumps.

PHASE 15 — TRANSLATION

Implement model movement.

Gesture:

```
Open palm + controlled hand movement
```

Map movement into the 3D world.

PHASE 16 — EXPLODED VIEW

This is one of the core features.

When the user performs:

```
Pull Apart
```

the selected assembly should separate.

Example:

```
NORMAL
[ENGINE]
EXPLODED
[HEAD]
      ↑
[PISTON]
      ↓
[CRANKSHAFT]
      ↓
[FLYWHEEL]
```

Calculate explosion vectors automatically based on:

- component center
- parent center
- object center

Each component should move smoothly.

Do not simply teleport objects.

Use interpolation.

PHASE 17 — MANUAL COMPONENT DISMANTLING

The user should be able to physically remove individual components.

Example:

User points to:

```
Battery
```

Pinches.

Pulls hand backward.

Battery moves outward.

Release.

Battery stays separated.

The application records:

```
Dismantled components:
1. Battery
2. Air Filter
3. Engine Cover
```

PHASE 18 — CONSTRAINT SYSTEM

Components should not behave like random floating objects.

Implement constraints.

Example:

A wheel should rotate around:

```
axle
```

A piston should move along:

```
cylinder axis
```

A door should rotate around:

```
hinge
```

A crankshaft should rotate around:

```
shaft axis
```

Create:

```
constraints.py
```

Support:

- fixed
- hinge
- slider
- rotational
- translational

PHASE 19 — MECHANICAL SIMULATION

Add interactive mechanical behavior.

Examples:

Engine

Rotate crankshaft.

Pistons move.

Connecting rods move.

Wheel

Rotate wheel.

Brake disc rotates.

Door

Open/close around hinge.

Suspension

Compress and release.

Gearbox

Rotate gears.

Move gear selector.

The goal is not full industrial-grade physics.

The goal is educational visualization.

PHASE 20 — REASSEMBLY

Implement reverse dismantling.

If the user selects:

```
Reassemble
```

components return to their original positions.

Use smooth animation.

Validate:

```
Correct component
Correct parent
Correct orientation
Correct position
```

PHASE 21 — STEP-BY-STEP LEARNING MODE

Create:

```
LEARNING MODE
```

Example:

Step 1

Remove engine cover.

Step 2

Remove spark plug.

Step 3

Remove cylinder head.

Step 4

Inspect piston.

Step 5

Inspect crankshaft.

Each step should include:

- instruction
- highlighted component
- required gesture
- explanation
- completion status

PHASE 22 — FREE PLAY MODE

Create:

```
FREE PLAY
```

In this mode the user can dismantle anything.

No predefined sequence.

Allow:

- select
- grab
- move
- rotate
- explode
- inspect
- reassemble

PHASE 23 — COMPONENT INFORMATION SYSTEM

When a component is selected, display:

```
COMPONENT
Name:
Spark Plug
Category:
Ignition
Function:
Produces the spark required to ignite the air-fuel mixture.
Connected Components:
Cylinder Head
Ignition Coil
Material:
Ceramic + Metal
Common Failure:
Electrode wear
Educational Level:
Beginner
```

Store information separately from rendering code.

PHASE 24 — MODEL LIBRARY

Create a model browser.

Categories:

```
AUTOMOTIVE
├── Cars
├── Motorcycles
├── Engines
└── Transmissions
ELECTRONICS
├── Smartphones
├── Laptops
├── GPUs
└── Motherboards
MECHANICAL
├── Gearbox
├── Hydraulic Pump
├── Robotics
└── Motors
```

The user should be able to select a model.

PHASE 25 — MODEL IMPORT SYSTEM

Allow developers to add models without changing Python code.

Example:

```
assets/models/
car_01/
    model.glb
    metadata.json
bike_01/
    model.glb
    metadata.json
phone_01/
    model.glb
    metadata.json
```

The application automatically detects available models.

PHASE 26 — FUTURISTIC HUD

Create a futuristic interface.

Display:

```
HOLOMECHANIX
MODEL:
SPORTS BIKE
MODE:
FREE PLAY
GESTURE:
PINCH
SELECTED:
ENGINE
FINGERS:
8 / 10
FPS:
59
TRACKING:
STABLE
```

Use a clean technical aesthetic.

Do not overcrowd the screen.

PHASE 27 — GESTURE VISUALIZATION

Display a small visualization showing detected hands.

Example:

```
LEFT HAND
● ● ● ● ●
RIGHT HAND
● ● ● ● ●
Gesture:
PINCH
Confidence:
94%
```

This will make debugging easier.

Allow developers to enable:

```
DEBUG MODE
```

PHASE 28 — VOICE COMMANDS

Optional feature.

Add voice commands such as:

```
"Open engine"
"Explode assembly"
"Select battery"
"Reassemble"
"Rotate left"
"Zoom in"
"Show components"
"Enter learning mode"
```

Use a suitable speech-recognition system.

Voice should supplement gestures, not replace them.

PHASE 29 — AI COMPONENT ASSISTANT

Add an optional AI assistant.

The assistant should answer:

```
What does this component do?
Why is this component connected here?
What happens if this component fails?
How does this mechanism work?
What should I remove next?
```

The AI must use the current model/component metadata as context.

Do not allow it to invent technical specifications.

Clearly distinguish:

```
Known model data
```

from:

```
General educational explanation
```

PHASE 30 — ADVANCED SPATIAL GESTURES

Implement advanced gestures.

Two-hand rotate
Rotate model.

Two-hand scale
Scale model.

Pinch + drag
Move component.

Pull apart
Explode assembly.

Push together
Reassemble.

Point
Select.

Double pinch
Confirm action.

Closed fist
Pause.

Open palm
Activate interaction.

Swipe
Change model.

PHASE 31 — GESTURE SAFETY SYSTEM

Never allow accidental destructive actions.

For major operations:

```
Pull apart
```

require:

```
gesture + stable detection
```

For example:

```
Pull Apart detected
Hold for 0.5 seconds
```

Then execute.

Display:

```
EXPLODE ASSEMBLY
████████▒▒ 80%
```

PHASE 32 — PERFORMANCE OPTIMIZATION

The application must target:

```
30+ FPS minimum
```

Prefer:

```
60 FPS
```

Optimize:

- MediaPipe processing
- rendering
- mesh complexity
- textures
- object updates
- gesture calculations

Use threading or asynchronous processing where appropriate.

Separate:

```
Camera thread
Vision processing
Rendering
UI
```

if supported by the architecture.

PHASE 33 — ERROR HANDLING

Handle:

- camera unavailable
- MediaPipe failure
- model missing
- corrupted model
- unsupported file format
- invalid metadata
- missing component
- rendering failure
- low tracking confidence

Never crash without a useful message.

PHASE 34 — TESTING

Create automated tests.

Test:

Vision

- finger counting
- gesture classification
- smoothing

Models

- model loading
- metadata validation
- hierarchy

Interaction

- selection
- grabbing
- movement
- rotation
- scaling

Dismantling

- explosion
- component removal
- reassembly

Safety

- accidental gesture prevention

PHASE 35 — CALIBRATION SYSTEM

Create a calibration screen.

User should:

1. Stand at appropriate distance.
2. Show both hands.
3. Open palms.
4. Move hands slowly.
5. System calculates tracking quality.

Display:

```
CAMERA
[▓▓▓▓▓▓▓▓▓]
LEFT HAND
[▓▓▓▓▓▓▓▓▒]
RIGHT HAND
[▓▓▓▓▓▓▓▓▓]
DEPTH ESTIMATION
[▓▓▓▓▓▓▓▒▒]
TRACKING QUALITY
92%
```

PHASE 36 — CAMERA SETUP GUIDE

Show users:

```
Recommended setup:
Camera:
720p or higher
Distance:
1–2 meters
Lighting:
Good front lighting
Background:
Low clutter
Hands:
Fully visible
```

Do not require special hardware for the basic version.

PHASE 37 — OPTIONAL DEPTH CAMERA SUPPORT

Design the architecture so it can later support:

- Intel RealSense
- depth cameras
- stereo cameras
- other supported sensors

If a depth camera is unavailable:

```
Fallback:
webcam + MediaPipe
```

Clearly label depth accuracy.

PHASE 38 — DATABASE

If persistent data is required, add a lightweight database.

Store:

- model metadata
- components
- user sessions
- learning progress
- completed lessons
- gesture configurations

SQLite is sufficient initially.

PHASE 39 — USER PROFILES

Optional.

Allow:

```
Guest
Student
Instructor
```

Students can save:

- completed models
- learning progress

- dismantling history

PHASE 40 — EDUCATIONAL QUIZ MODE

After dismantling a model, generate questions.

Example:

```
Which component converts chemical energy
into mechanical energy?
A. Battery
B. Engine
C. Radiator
D. Brake
```

The user answers using:

- hand gesture
- mouse
- keyboard

PHASE 41 — RECORDING MODE

Allow the user to record an interaction session.

Record:

- selected model
- gestures
- selected components
- dismantling sequence

Allow replay.

PHASE 42 — SCREENSHOT / EXPORT

Allow:

```
Screenshot
```

and optionally:

```
Export exploded view
```

Possible output:

```
PNG
```

or:

```
3D scene
```

PHASE 43 — MODERN APPLICATION UI

Create a professional interface.

Main screen:

```
┌──────────────────────────────────────┐
│ HOLOMECHANIX                FPS 59   │
├──────────────────────────────────────┤
│                                      │
│                                      │
│              3D MODEL                │
│                                      │
│                                      │
│                                      │
├──────────────────┬───────────────────┤
│ MODEL            │ COMPONENT         │
│ SPORTS BIKE      │ ENGINE            │
│                  │                   │
│ MODE             │ FUNCTION          │
│ FREE PLAY        │ ...               │
└──────────────────┴───────────────────┘
```

PHASE 44 — WEB VERSION

If the architecture supports a browser frontend:

Backend:

```
FastAPI
```

Frontend:

```
React
Three.js
WebGL
```

Communication:

```
WebSocket
```

Flow:

```
Camera
   ↓
Python
   ↓
MediaPipe
   ↓
Gesture Engine
   ↓
WebSocket
   ↓
Three.js
   ↓
3D Scene
```

This architecture is recommended for a polished deployment.

PHASE 45 — API DESIGN

Create APIs such as:

```
GET /models
GET /models/{id}
GET /models/{id}/components
GET /components/{id}
POST /session
POST /gesture
GET /learning/{model_id}
```

Use WebSockets for real-time interaction rather than REST for every frame.

PHASE 46 — SECURITY

Do not expose:

- API keys
- secrets
- database credentials

Use:

```
.env
```

and:

```
.env.example
```

Add secrets to:

```
.gitignore
```

PHASE 47 — LOGGING

Create structured logs.

Example:

```
[INFO] Camera initialized
[INFO] MediaPipe initialized
[INFO] Model loaded: sports_bike
[INFO] Gesture detected: PINCH
[INFO] Component selected: Engine
[INFO] Component detached: Engine
```

PHASE 48 — DOCUMENTATION

Create a complete:

```
README.md
```

Include:

- project overview
- architecture
- installation
- requirements
- camera setup
- controls
- gesture list
- model format
- metadata format
- adding new models
- troubleshooting
- development
- deployment

Also create:

```
docs/
```

with:

```
architecture.md
gesture-system.md
model-system.md
dismantling-system.md
deployment.md
```

PHASE 49 — DEPLOYMENT ARCHITECTURE

IMPORTANT:

A normal cloud server cannot directly access the user's local webcam in the same way as a local Python OpenCV application.

Therefore design deployment correctly.

Local desktop version

Best for:

- OpenCV
- MediaPipe
- GPU rendering
- low latency

Package using:

```
PyInstaller
```

Create:

```
HoloMechanix.exe
```

where feasible.

PHASE 50 — WEB DEPLOYMENT

For browser deployment:

Frontend:

```
Vercel
```

Backend:

```
Render
Railway
AWS
Azure
Google Cloud
```

or another suitable service.

Camera access happens in the browser using:

```
getUserMedia()
```

Gesture processing can happen:

```
client-side
```

or:

```
backend
```

Prefer client-side processing when possible to reduce latency and privacy concerns.

PHASE 51 — DOCKER

Create:

```
Dockerfile
docker-compose.yml
```

Support:

```
docker compose up
```

The containerized deployment should be suitable for the backend/API.

Document hardware limitations for GPU-dependent rendering.

PHASE 52 — CI/CD

Create GitHub Actions.

Pipeline:

```
Push
 ↓
Install dependencies
 ↓
Run tests
 ↓
Lint
 ↓
Build
 ↓
Deploy
```

PHASE 53 — FINAL DEMO MODEL

After the core system works, create a polished demonstration model.

Recommended:

```
Sports Motorcycle
```

The demo should contain:

```
Frame
Engine
Fuel Tank
Battery
Air Filter
Spark Plug
Exhaust
Transmission
Chain
Front Wheel
Rear Wheel
Brakes
Suspension
```

The user should be able to dismantle it interactively.

PHASE 54 — FINAL USER EXPERIENCE

The final experience should follow this flow:

```
START
  ↓
Camera Permission
  ↓
Calibration
  ↓
Main Laboratory
  ↓
Select Model
  ↓
Load 3D Model
  ↓
Detect Hands
  ↓
Recognize Gesture
  ↓
Select Component
  ↓
Grab Component
  ↓
Move Component
  ↓
Dismantle
  ↓
Inspect
  ↓
Learn
  ↓
Reassemble
  ↓
Complete
```

PHASE 55 — ADVANCED FUTURE FEATURES

Design the architecture so the following can be added later:

- AR support
- VR support
- Apple Vision Pro style interaction
- Meta Quest support
- Leap Motion support
- Ultraleap
- RealSense
- multi-user collaboration
- multiplayer engineering laboratory
- AI tutor
- automatic model decomposition
- automatic component labeling
- CAD integration
- physics simulation
- realistic material simulation
- voice assistant
- eye tracking

- holographic display support

Do not implement these unless requested.

Design interfaces so they can be added later.

56. CRITICAL TECHNICAL LIMITATIONS

The system must NOT falsely claim that a normal webcam provides true holography.

Explain the distinction:

```
Tony Stark-style holography
        ≠
normal webcam
```

The project should instead create a:

```
spatial 3D interaction illusion
```

using:

- hand tracking
- 3D rendering
- depth estimation
- gesture recognition
- animated objects
- perspective
- lighting
- HUD effects

If actual volumetric hardware is unavailable, do not call the output a true hologram.

Call it:

```
Interactive Spatial 3D Interface
```

57. IMPORTANT DESIGN PRINCIPLE

Do not make the application look like:

```
OpenCV window
+
3D object
+
some text
```

It should feel like an engineering laboratory.

Visual language:

- dark environment
- subtle cyan/white technical UI
- thin lines
- spatial labels
- floating information panels
- component callouts
- smooth transitions
- minimal HUD
- realistic 3D models

- responsive interaction
- clean typography

Avoid unnecessary neon overload.

58. PERFORMANCE TARGETS

Target:

```
Camera:
30–60 FPS
Gesture latency:
<100 ms where hardware permits
Interaction:
smooth
Model loading:
progress indicator
Gesture confidence:
visible in debug mode
```

The application must degrade gracefully on weaker hardware.

59. CODE QUALITY

Follow:

- PEP 8
- type hints
- modular architecture
- docstrings
- error handling
- logging
- reusable classes
- configuration files
- environment variables

Do not put the entire project in:

```
main.py
```

60. FINAL PROJECT VALIDATION

Before declaring the project complete, verify:

Computer Vision

- ■ Camera works
- ■ One hand works
- ■ Two hands work
- ■ 10 fingers detected
- ■ Gestures stable
- ■ Tracking survives movement
- ■ Calibration works

3D

- ■ Model loads
- ■ Model rotates
- ■ Model scales
- ■ Model moves
- ■ Components selectable
- ■ Component highlighting works

Dismantling

- ■ Component can be grabbed
- ■ Component can be detached
- ■ Exploded view works
- ■ Components respect constraints
- ■ Reassembly works

Education

- ■ Component information works
- ■ Learning mode works
- ■ Quiz works
- ■ Progress tracking works

Application

- ■ Error handling
- ■ Logging
- ■ Configuration
- ■ Tests
- ■ Documentation
- ■ Deployment
- ■ GitHub repository
- ■ Release build

61. DEVELOPMENT EXECUTION RULE

Follow this exact progression:

```
PHASE 0   Environment
PHASE 1   Camera
PHASE 2   Hand tracking
PHASE 3   10 fingers
PHASE 4   Gestures
PHASE 5   Gesture stabilization
PHASE 6   Spatial mapping
PHASE 7   3D engine
PHASE 8   First model
PHASE 9   Component hierarchy
PHASE 10  Selection
PHASE 11  Highlighting
PHASE 12  Grabbing
PHASE 13  Rotation
PHASE 14  Scaling
PHASE 15  Translation
PHASE 16  Exploded view
PHASE 17  Dismantling
PHASE 18  Constraints
PHASE 19  Mechanical simulation
```

```
PHASE 20  Reassembly
PHASE 21  Learning mode
PHASE 22  Free play
PHASE 23  Component information
PHASE 24  Model library
PHASE 25  Model importer
PHASE 26  HUD
PHASE 27  Gesture visualization
PHASE 28  Voice
PHASE 29  AI assistant
PHASE 30  Advanced gestures
PHASE 31  Safety
PHASE 32  Optimization
PHASE 33  Error handling
PHASE 34  Testing
PHASE 35  Calibration
PHASE 36  Camera setup
PHASE 37  Depth camera support
PHASE 38  Database
PHASE 39  Profiles
PHASE 40  Quiz
PHASE 41  Recording
PHASE 42  Export
PHASE 43  UI
PHASE 44  Web version
PHASE 45  API
PHASE 46  Security
PHASE 47  Logging
PHASE 48  Documentation
PHASE 49  Desktop deployment
PHASE 50  Web deployment
PHASE 51  Docker
PHASE 52  CI/CD
PHASE 53  Final demo model
PHASE 54  Final UX
PHASE 55  Future architecture
FINAL RELEASE
```

62. HOW TO WORK WITH ME

I am a beginner/intermediate developer.

Do not assume I already know the required setup.

For each implementation stage:

1. Tell me exactly what we are building.
2. Explain why we need it.
3. Give me the exact folder/file to create.
4. Give me the exact code.
5. Give me the exact command to run.
6. Tell me what output I should see.
7. Tell me how to test it.
8. Tell me common errors and their fixes.
9. Wait for me to confirm that the stage works.
10. Only then move to the next stage.

Do NOT give me 10 implementation tasks at once.

Give me ONE actionable task at a time.

When I reply:

```
DONE
```

continue to the next task.

If something fails, stop advancing and debug that stage first.

63. FIRST RESPONSE REQUIREMENT

Do NOT start implementing the entire application immediately.

Your first response should:

1. Explain the overall architecture.
2. Explain the difference between true holography and our spatial 3D interface.
3. Explain the recommended technology stack.
4. Explain the development roadmap.
5. Check the available Python version and environment.
6. Begin ONLY with Phase 0.
7. Give me the first setup task.

Do not provide future implementation code until the current task is completed.

The goal is to build a real working product incrementally, not a mockup.

FINAL OBJECTIVE

Build **HoloMechanix** into a genuinely interactive spatial engineering platform where a user can stand in front of a camera, use both hands and 10 fingers, select and manipulate 3D objects, dismantle their components, study their internal structure, observe mechanical behavior, and reassemble them.

The core experience should feel like:

```
USER
  ↓
10 FINGERS
  ↓
CAMERA
  ↓
COMPUTER VISION
  ↓
GESTURE ENGINE
  ↓
SPATIAL INTERACTION
  ↓
3D ENGINE
  ↓
COMPONENT SYSTEM
  ↓
MECHANICAL SIMULATION
  ↓
EDUCATIONAL EXPERIENCE
```

Build it professionally, modularly, incrementally, and realistically.